DOCENTE	Massimo Orengia
UNITA' FORMATIVA	Copyright e norme giuridiche del mondo digitale
ARGOMENTO	Slides Cloud Specialist – seconda lezione

IaaS, PaaS e SaaS

sono i tre modelli principali di servizi **cloud computing** che offrono diversi livelli di gestione e controllo:

- **IaaS** (Infrastructure as a Service) fornisce risorse di base (server, storage, rete),
- **PaaS** (Platform as a Service) offre un ambiente per sviluppare e distribuire app (infrastruttura + piattaforma),
- **SaaS** (Software as a Service) offre un'applicazione completa pronta all'uso, accessibile via web, gestita interamente dal provider

IaaS, PaaS, SaaS differenze



Capterra

Caratteristica	IaaS (Infrastructure)	PaaS (Platform)	SaaS (Software)
Esempio AWS	Amazon EC2	AWS Elastic Beanstalk	Amazon QuickSight
Cosa gestisce AWS	Hardware, Rete, Virtualizzazione.	Runtime, Middleware, OS.	Tutta l'applicazione.
Cosa gestisci TU	OS, Middleware, Dati, App.	Codice e Dati.	Solo i Dati e l'Accesso.
Responsabilità Legale	Massima (Configurazione).	Media (Codice).	Minima (Utilizzo).
Flessibilità	Totale.	Alta.	Limitata.

Il Modello di Responsabilità Condivisa

- **Concetto chiave:** AWS è responsabile della sicurezza **DEL** cloud (Hardware, Global Infrastructure).
- **Il Cliente** è responsabile della sicurezza **NEL** cloud (Dati, Crittografia, Configurazioni).

Analisi Contrattuale AWS - I 3 Pilastri

1. **AWS Customer Agreement:** Il contratto base (termini di utilizzo, pagamenti, proprietà intellettuale).
2. **Service Level Agreements (SLA):** Le promesse di uptime (es. 99.99%). Se AWS cade, hai diritto a rimborsi (Service Credits).
3. **Service Terms:** Regole specifiche per ogni servizio (es. "Non puoi usare il servizio X per scopi militari").

IaaS (Amazon EC2) - "Affitto il Garage"

- **Punto contrattuale:** AWS garantisce che l'hardware funzioni.
- **Responsabilità normativa:** Tu devi fare il patching dell'OS. Se non lo fai e subisci un attacco, hai violato il principio di "Accountability" del GDPR.
- **Copyright:** Tutto il software che installi sopra è di tua proprietà o licenza.

PaaS (Elastic Beanstalk / Lambda) - "Affitto la Cucina"

- **Punto contrattuale:** AWS gestisce il sistema operativo. Tu ti occupi solo del codice.
- **Vantaggio legale:** Meno oneri di manutenzione tecnica, ma resta la responsabilità sul trattamento dei dati (Database, API).

SaaS (QuickSight / Chime) - "Vado al Ristorante"

- **Punto contrattuale:** Tutto è gestito da AWS.

- **Responsabilità:** Ti occupi solo di chi ha le password per entrare. È il modello più semplice ma con meno controllo sui dati grezzi.

IaaS (Infrastructure as a Service)

Vantaggi	Svantaggi
Controllo Totale: Hai accesso alla root del sistema; puoi configurare tutto come desideri.	**Onere di Gestione:** Devi occuparti di aggiornamenti (patch), sicurezza del sistema operativo e backup.
Scalabilità Dinamica: Aggiungi o togli risorse (RAM, CPU) in pochi secondi.	**Complessità:** Richiede competenze tecniche elevate per l'amministrazione di sistema.
Costi su Consumo: Paghi solo per il tempo in cui l'infrastruttura è accesa.	**Responsabilità Legale:** Sei l'unico responsabile se il sistema operativo non è protetto.

Paas (Platform as a Service)

Vantaggi	Svantaggi
Focus sul Codice: Gli sviluppatori non perdono tempo a gestire server o database sottostanti.	**Lock-in del Fornitore:** Spostare un'applicazione da un PaaS all'altro (es. da AWS a Azure) può essere difficile.
Deployment Rapido: Il software va "live" molto più velocemente rispetto allo IaaS.	**Meno Personalizzazione:** Non puoi modificare i parametri profondi del sistema operativo o del runtime.
Aggiornamenti Automatici: AWS gestisce la sicurezza e le patch del middleware/OS per te.	**Costi Imprevedibili:** Se il codice non è ottimizzato, i costi di esecuzione possono scalare rapidamente.

Saas (Software as a Service)

Vantaggi	Svantaggi
Pronto all'uso: Nessuna installazione o configurazione tecnica necessaria.	**Controllo Zero:** Non hai accesso all'infrastruttura; se il servizio va giù, puoi solo aspettare.
Accesso Universale: Funziona via browser da qualsiasi dispositivo e luogo.	**Problemi di Integrazione:** Potrebbe essere difficile far comunicare il software con altri sistemi aziendali.
Zero Manutenzione: L'utente finale non deve preoccuparsi di bug o aggiornamenti.	**Privacy dei Dati:** I tuoi dati risiedono interamente sui server del fornitore; devi fidarti dei loro protocolli.

LA LICENZA

La **licenza** è

un **contratto** con cui il titolare dei diritti concede ad altri

certe **facoltà** (installare, usare, copiare, modificare, ridistribuire)

e pone **limiti** (niente reverse engineering, niente ridistribuzione, obbligo di rilasciare il codice modificato, ecc.).

Nel mondo IT esistono numerosi tipi di licenze, che si possono raggruppare in due macro-categorie:

Proprietarie (codice chiuso, come Freeware, Shareware, EULA)

Libere/Open Source, quest'ultime ulteriormente suddivise in:

- **Non-Copyleft** (MIT, BSD),
- **Copyleft Debole** (LGPL, MPL),
- **Copyleft Forte** (GPL)
- **Network Copyleft** (AGPL).

Il termine **COPYLEFT** è un gioco di parole: *right* (diritto) diventa *left* (lasciato).

Le licenze variano anche in base al modello di pagamento (perpetua, sottoscrizione, pay-per-use) e al tipo di utente (singolo, gruppo, aziendale).

licenza proprietaria

- Il codice è in genere **non accessibile** (solo binari/eseguibili).

- L'autore/titolare **mantiene il massimo controllo**: l'utente ottiene solo un **diritto d'uso limitato** (es. installazione su PC, per un certo periodo, per un certo scopo).
- Le **modifiche** sono in genere vietate o fortemente limitate; spesso è vietata la **redistribuzione** a terzi.

- Il modello economico principale è: **paghi la licenza o l'abbonamento**; in cambio hai il diritto di usare il software come previsto dal contratto.

In sintesi: **"tutti i diritti riservati, concedo solo quello che è scritto nel contratto (EULA)"**.

Principali tipi di licenze proprietarie

4.1 Modelli base dal punto di vista contrattuale

1. Licenza d’uso standard / EULA (End User License Agreement)

- Contratto “clicca per accettare” mostrato all’installazione o al primo avvio.
- Concede in genere un **diritto non esclusivo e non trasferibile** di usare il software.
- Può imporre: divieto di reverse engineering, di copia non autorizzata, di uso oltre certe condizioni.
-

2. Licenza per postazione (per device)

- Una licenza per **singolo computer** o device (es. 1 licenza = 1 PC).
- Spesso per uffici e PMI.

3. Licenza per utente (user based)

- Una licenza per **utente nominativo**, che può usare il software su più device.
- Tipica dei software aziendali, CRM, gestione documentale.

4. Licenza per server / per core / per istanza

- Tariffazione in base al **numero di server**, o ai **core**, o alle **istanze** di esecuzione.
- Tipica dei software di database o middleware aziendali.

5. Licenze a tempo (abbonamento / subscription)

- Il diritto d'uso dura finché paghi un **canone periodico** (mensile/annuale).

6. SaaS (Software as a Service)

- Non installi localmente il software, lo usi via cloud.

- La licenza regola l'**accesso al servizio** (numero utenti, API, limiti di uso), non solo l'uso del programma installato.

7. Shareware

- Software proprietario che puoi **provare gratuitamente** (in versione completa o limitata) per un certo periodo o con alcune funzioni ridotte.
  Dopo il periodo di prova o se vuoi funzioni complete devi **acquistare la licenza**.

8. Trial

- Versione di prova **a tempo** (es. 14 o 30 giorni), dopo la quale il software smette di funzionare se non acquisti la licenza.

9. Freemium

- Una parte delle funzionalità è **gratuita** (spesso con pubblicità o limitazioni).
- Funzionalità avanzate richiedono **pagamento** o abbonamento.

10. OEM (Original Equipment Manufacturer)

- Licenza venduta **insieme all'hardware** (es. sistema operativo preinstallato sul PC).
- Di solito legata a **quella macchina** e con limiti di trasferimento.

licenza Open Source

- Il software resta **protetto dal diritto d'autore**, ma l'autore sceglie di **concedere ampie libertà** tramite la licenza.

- Regola tipica Open Source: l'utente può **accedere al codice sorgente**, **studiare**, **modificare**, **redistribuire** il software, anche **a fini commerciali**, nei limiti e alle condizioni poste dalla singola licenza (copyleft forte, debole, permissiva).

- L'autore usa il diritto d'autore per **garantire libertà**, non per limitarle: la licenza impone soprattutto **obblighi di trasparenza** (es. mantenere il copyright notice) e, nel copyleft, l'**obbligo di rilasciare il codice modificato con la stessa licenza**.

In sintesi: **"software libero/Open Source: l'autore permette e protegge la libertà di usare, studiare, modificare e ridistribuire il software, a certe condizioni"**.

- **Non-Copyleft:** Permettono di integrare il codice in software proprietario (es. MIT, BSD, Apache).

- **Copyleft Debole:** Permettono l'integrazione in software proprietario ma richiedono che le modifiche alla libreria siano rilasciate sotto la stessa licenza (es. LGPL, MPL).

- **Copyleft Forte:** Impongono che qualsiasi software derivato venga rilasciato con la stessa licenza (es. GPL).

- **Network Copyleft:** Estendono il copyleft forte anche all'uso in rete (es. AGPL).

licenze Open Source e obblighi principali

1. Copyleft forte

- Logica: se ridistribuisci il software (anche modificato), devi **rilasciare il codice sorgente** e usare la **stessa licenza** o una compatibile molto vicina.

- Effetto pratico: il codice “contagia” i progetti in cui viene integrato in modo forte (soprattutto se linkato staticamente o parte integrante dell’applicazione).

- Esempi:

  GPL (GNU General Public License) – molto diffusa per applicazioni

  - **AGPL (Affero GPL)** – estende il copyleft anche ai servizi web: anche se modifichi e usi il software per offrire un servizio in rete, devi rendere disponibile il codice modificato.

2. Copyleft debole (weak copyleft)

- Logica: il copyleft si applica **soprattutto alla libreria** o ai file modificati, ma consente che il software proprietario si **colleghi** (linki) a quella libreria senza diventare automaticamente coperto dall'intera GPL.

- Esempi:

  - **LGPL (Lesser GPL)** – pensata per librerie: chi usa la libreria può tenere proprietario il proprio codice, ma se modifica la libreria deve rilasciare quelle modifiche come LGPL/GPL.

  - **MPL (Mozilla Public License)** – copyleft a **livello di file**: i file modificati restano sotto MPL, ma puoi combinare il progetto con codice proprietario in altri file.

3. Licenze permissive

- Logica: massimo grado di libertà; permettono di **prendere il codice, modificarlo, ridistribuirlo anche dentro software proprietario**, spesso senza obbligo di rilasciare il sorgente.

- Obblighi tipici: mantenere le **note di copyright** e le **clausole di esclusione di responsabilità**.

- Esempi:

    - **MIT License** – molto breve e permissiva; basta mantenere il copyright notice.
      BSD (2 clause,3 clause) – simile a MIT, con lievi variazioni sulle clausole di pubblicità/responsabilità.
      Apache 2.0 – permissiva ma più strutturata, include clausole sui brevetti (patent grant) e condizioni specifiche sull'uso dei marchi.

proprietario vs Open Source

1. Dal punto di vista dell'autore/titolare

- **Licenza proprietaria**

  - Mantiene il controllo massimo.
    Definisce contrattualmente esattamente cosa può fare l'utente.
    Spesso usa modelli commerciali a pagamento (licenza, abbonamento).

- **Licenza Open Source**

  - Usa il diritto d'autore per concedere **ampie libertà**.
    Impone soprattutto obblighi di **trasparenza** e, nelle licenze copyleft, di **condivisione del codice derivato**.
    Modelli economici: servizi, supporto, personalizzazioni, dual licensing.

2. Dal punto di vista dell’utente / sviluppatore

- **Con software proprietario**
  - Puoi usare il software **solo nei limiti** della licenza (numero utenti/device, durata)
  - In genere **non puoi studiare il codice sorgente** (non disponibile). 4
  - Non puoi modificare e ridistribuire liberamente.
- **Con software Open Source**
  - Puoi **accedere al sorgente**, studiarlo, modificarlo.
  - Puoi ridistribuire (con o senza modifiche), ma devi **rispettare gli obblighi** specifici (copyleft / note di copyright / condizioni su brevetti).